SmartTest

AI-Powered Code Coverage and Test Automation Tool

Technical Specification and Working Guide
Prepared as the build brief for Devin AI

How to use this document. This is the complete specification for building SmartTest as a web application: a FastAPI backend, a React frontend, and a local open-source LLM served by Ollama. It explains what the product does, how every step of the pipeline works, the exact rules for classifying generated tests, the API, the data shapes, the screens, the four headline UI features, and a phased build plan with acceptance criteria. Read all of it before writing code. Where this document gives a rule, follow it. Where it leaves a detail open, choose the simplest option that works and note the choice in the README.

Non-negotiables. Everything must run locally and free of cost. No paid APIs. The LLM is reached through a pluggable client whose default is Ollama. Generated tests are never trusted until they have been executed and classified.

1. Product Overview

1.1 The problem

Developers are expected to write unit tests, but under deadline pressure many functions are never tested. Nobody knows whether those functions work, so a later change can break them silently. Writing the missing tests by hand is slow and repetitive.

1.2 What SmartTest does

SmartTest takes a Python project, finds the functions that have no (or poor) test coverage, ranks them by risk, asks a local LLM to write pytest tests for the riskiest ones, runs those tests, and classifies every result. If a well-formed test fails against the real function, SmartTest flags a **Possible Bug Found**. At the end it shows how much coverage improved and lets the user download the report and the accepted tests.

1.3 Target users

- Development teams and students with untested or legacy Python code.
- Open-source maintainers who want to find coverage gaps quickly.
- QA engineers who want a first draft of tests to review instead of writing from scratch.

1.4 Scope

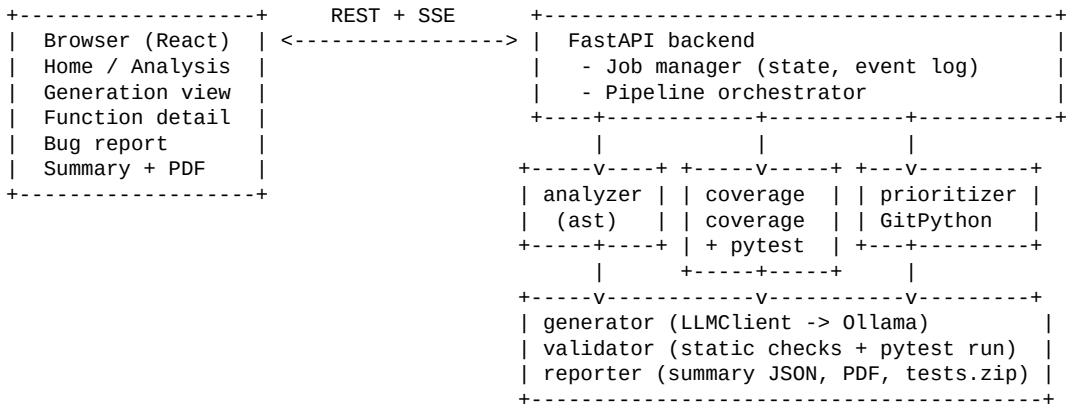
In scope (version 1)	Out of scope (version 1)
Python projects only (zip upload or GitHub URL)	Other languages
Coverage gap detection, risk ranking, LLM test generation, execution, classification	Automatically fixing the user's source code
Bug Discovery Highlight	CI/CD integration, user accounts, multi-user hosting
Web UI with side-by-side view, live progress feed, summary cards, PDF export	Mutation testing badge (optional stretch goal only, see section 14)

2. Technology Stack (fixed)

Layer	Choice	Notes
Backend language	Python 3.11+	Single codebase for analysis, generation and the API
Web framework	FastAPI + Uvicorn	REST endpoints plus Server-Sent Events (SSE) for the live feed
Frontend	React 18 + Vite + TypeScript + Tailwind CSS	Clean, modern, desktop-first, responsive
Syntax highlighting	react-syntax-highlighter (Prism)	Used in the side-by-side code view
Code parsing	Python `ast` module	Function extraction and call-site counting
Coverage	coverage.py (branch mode off by default)	Run through subprocess, read `coverage json`
Git history	GitPython (plus `git blame --porcelain`)	Recency of last change per function
LLM	Ollama HTTP API at `http://localhost:11434`	Default model `qwen2.5-coder:7b`; `codellama:7b`, `llama3.1:8b` also supported. Model is a setting.
Test runner	pytest (+ `pytest-json-report` or JUnit XML)	Run in a subprocess with a timeout
PDF export	reportlab	Generated by the backend, downloaded by the browser
Storage	JSON files under `data/jobs/<job_id>/`	No database needed for version 1
Tests for SmartTest itself	pytest (backend), Vitest (frontend, light)	See build phases

LLM client abstraction. Implement an `LLMClient` interface with `generate(prompt) -> str`. Provide two implementations: `OllamaClient` (default) and `MockLLMClient` (returns canned, valid pytest code for the sample repository, used by automated tests and for development on machines without a GPU). Select with the environment variable `LLM_PROVIDER=ollama|mock`.

3. System Architecture



Each job gets an isolated workspace folder. The backend never runs the user's code or generated tests inside its own process; all execution happens in subprocesses with timeouts (section 11).

4. How the Project Works, Step by Step

The product has two user-triggered phases. **Phase A (Analyze)** runs automatically after the user submits a project. **Phase B (Generate)** runs when the user chooses functions and clicks Generate. Both phases stream messages to the live progress feed.

Step 0. Input and job creation

- The user either uploads a `.zip` of a Python project or pastes a public GitHub URL, and optionally changes settings (model name, number of functions to process, coverage threshold).
- `POST /api/jobs` creates a job id, creates `data/jobs/<id>/repo/`, extracts the zip or runs a full `git clone` (not shallow, because history is needed for recency scoring).
- Validate: at least one `.py` file, upload under `MAX_UPLOAD_MB` (default 50), reject path traversal in zip entries (zip-slip).
- The browser opens `GET /api/jobs/{id}/events` (SSE) and the analysis starts in a background task.

Step 1. Code structure analysis (analyzer)

- Walk all `.py` files. Skip `.git`, `venv`, `.venv`, `node_modules`, `__pycache__`, `site-packages`, and any test files (`tests/`, `test_*.py`, `*_test.py`, `conftest.py`). Test files are used only for coverage.
- Parse each file with `ast.parse`. For every `FunctionDef` and `AsyncFunctionDef` (including methods, with qualified name `Class.method`) record: id, file, name, qualname, line start and end, argument names, defaults, type annotations, return annotation, docstring, decorators, full source text, and whether it is a method.
- Skip functions that are trivial: body is only `pass`, `...` or a docstring; simple `__repr__`/`__str__`; abstract methods. Keep `__init__` only if it contains logic beyond plain attribute assignment.
- Also record the file's top-level import lines and, for methods, the class's `__init__` signature. The generator needs this context.
- Files with syntax errors are skipped and reported as a warning in the feed, not as a crash.

Step 2. Baseline coverage (coverage_check)

- Try to install the project's dependencies into a per-job virtual environment if `requirements.txt` exists (timeout 120 s). If it fails, continue and warn.
- Write a `.coveragerc` with `source = .` and `omit` for test files so that files never imported still appear at 0 percent.
- Run `coverage run -m pytest -q` in the repo workspace, then `coverage json -o coverage.json`. If the project has no tests, pytest collects nothing and every statement is reported as missing, which is correct.
- Map line data to functions: for each function, `covered = executed lines inside its range` and `function_coverage_pct = covered / statements_in_range * 100`.
- Candidates for generation are functions with `function_coverage_pct < COVERAGE_THRESHOLD` (default 60). Functions at 0 percent are labelled **Untested**, the others **Partially tested**.
- Store `coverage_before` (total percent) and per-function percentages.

Step 3. Risk-based prioritization (prioritizer)

Each candidate gets a risk score between 0 and 1 from two signals:

Signal	How to compute	Score
Recency of change	Run `git blame -L start,end --porcelain HEAD -- file`, take the newest committer-time among the function's lines; `days = now - that time`. If the project has no git history use the file's modified time.	`recency = 1 / (1 + days/30)`

Signal	How to compute	Score
Usage frequency	Count call sites of the function name (`ast.Call` with matching `Name` or `Attribute`) across all non-test source files, excluding its own definition.	`usage = min(calls / max_calls_among_candidates, 1)`
Risk score	Equal weights, configurable	`risk = 0.5*recency + 0.5*usage`

- Labels: High if risk is at least 0.66, Medium if at least 0.33, otherwise Low.
- Sort candidates by risk descending. The Analysis Dashboard shows all candidates; the generator defaults to the top `max_functions` (default 10).
- Store for each function: `last_modified_days`, `usage_count`, `risk_score`, `risk_label`.

When Step 3 finishes, emit `analysis_complete`. The user sees the Analysis Dashboard (summary cards and the ranked table) and can select functions and press **Generate Tests**.

Step 4. AI test generation (generator)

- `POST /api/jobs/{id}/generate` with `{"function_ids": [...]}` or `{"all": true}` starts Phase B. Functions are processed one at a time (a local LLM is the bottleneck).
- For each function build the prompt from the template in section 9, including the exact import statement (computed from the file path, handling a `src/` layout), the function source, the docstring, the class context for methods, and up to 2 example call sites from the codebase.
- Call the LLM. Extract the first fenced Python code block with a regex. If there is none, treat the whole reply as code.
- Run the static checks of Step 5 immediately. If they fail, retry up to 2 times with the retry prompt, which includes the previous code and the specific error.
- Write the test to `data/jobs/<id>/generated_tests/test_<module>__<function>.py`. Never write into the user's own `tests/` folder.

Step 5. Validation and classification (validator)

Every generated test goes through two gates: **static checks**, then **execution**. Evaluate in this order.

Gate 1: static checks with ast (no execution)

Check	Rule	On failure
C1 Valid syntax	`ast.parse` succeeds	Retry, then REJECTED
C2 Targets the function	Imports the target and at least one test function calls it (directly or through an instance of its class)	Retry, then REJECTED (reason: does not call target)
C3 Meaningful assertion	At least one `assert` or `pytest.raises` involving the call result. Reject `assert True`, `assert 1 == 1`, or asserts on constants only	Retry, then REJECTED (reason: trivial test)

Gate 2: execution with pytest

Run the test file in a subprocess (timeout `TEST_TIMEOUT_SEC`, default 30) with the repo root and `src/` on `PYTHONPATH`. Parse the result per test function and apply these rules, in order:

Observed result	Classification	Action
Collection error: ImportError, ModuleNotFoundError, NameError, fixture error, wrong number of arguments passed at the test's call site	**REJECTED** (invalid test)	Retry up to 2 times with the error fed back; then reject with reason
All tests pass	**ACCEPTED**	Keep the test

Observed result	Classification	Action
`AssertionError` in a test that passed Gate 1, or an exception raised from inside the target function on plausible inputs	Goes to the **bug verification step** below	Rerun once to rule out flakiness
Timeout, or result differs between two runs (flaky)	**NEEDS REVIEW**	Show the reason
Bug verification verdict `likely_bug`	**POSSIBLE BUG FOUND**	Store expected vs actual, traceback (trimmed), and the LLM's one-line reason
Verdict `test_wrong`	Regenerate once with feedback; if it fails again, **REJECTED**	Never show as a bug
Verdict `unclear`	**NEEDS REVIEW**	Let the human decide

Bug verification step

To avoid false bug claims, when a well-formed test fails, ask the LLM the verification question in section 9 (function, docstring, failing test, pytest output) and expect JSON `{"verdict": "likely_bug|test_wrong|unclear", "reason": "..."}`. Parse defensively; if the reply is not valid JSON, treat it as `unclear`. This is what separates a real **Possible Bug Found** from an AI that simply guessed the wrong expected value.

Function-level status

A function card shows one status. Precedence: POSSIBLE BUG FOUND, then NEEDS REVIEW, then ACCEPTED (all tests pass), then REJECTED (every attempt was invalid). Individual test results are still listed inside the card.

Step 6. After-coverage

- Collect all ACCEPTED test files into `accepted_tests/`.
- Re-run `coverage run -m pytest` on the user's original tests plus `accepted_tests/` (do not include failing or rejected tests).
- Store `coverage_after`, per-function after-percentages, and the list of functions that moved from Untested or Partially tested to Tested.

Step 7. Report and outputs

- Build `summary.json` (shape in section 7) and emit `generation_complete`.
- The UI shows the Summary Report: summary cards, before/after coverage, per-function results, the bug report.
- `GET /api/jobs/{id}/report.pdf` builds the PDF with reportlab. `GET /api/jobs/{id}/tests.zip` downloads the accepted tests.

5. Feature List

#	Feature	Type
1	Upload a zip or paste a GitHub URL as input	Core
2	AST-based function extraction	Core
3	Coverage gap detection (Untested / Partially tested)	Core
4	Risk-based prioritization (recency + usage)	Core
5	LLM test generation with retries (Ollama, pluggable client)	Core
6	Automated validation and classification (Accepted / Needs Review / Rejected)	Core
7	**Bug Discovery Highlight** (Possible Bug Found, with verification step)	Core
8	Before/after coverage report	Core
9	**Side-by-side code view** with syntax highlighting	UI feature
10	**Live progress feed** over SSE	UI feature
11	**PDF report export**	UI feature
12	**Summary cards**	UI feature
13	Download accepted tests as a zip	Core

6. Backend API

Method and path	Purpose
`GET /api/health`	Returns backend status, whether Ollama is reachable, and the configured model
`GET /api/models`	Lists installed Ollama models (proxy of `/api/tags`)
`POST /api/jobs`	Multipart: `file` (zip) or `repo_url`, plus `settings` JSON. Returns `{job_id}` and starts Phase A
`GET /api/jobs/{id}`	Job state: stage, status, counts, settings
`GET /api/jobs/{id}/events`	SSE stream of events (section 8). Supports `Last-Event-ID` so the page can reconnect and replay
`GET /api/jobs/{id}/functions`	All extracted functions with coverage, risk and status
`GET /api/jobs/{id}/functions/{fid}`	One function: source, generated tests, per-test results, errors, bug details
`POST /api/jobs/{id}/generate`	Body `{function_ids: [...]}` or `{all: true}`. Starts Phase B
`GET /api/jobs/{id}/summary`	Summary card values and before/after coverage
`GET /api/jobs/{id}/report.pdf`	PDF report download
`GET /api/jobs/{id}/tests.zip`	Accepted tests download

Enable CORS for the Vite dev origin. Return clear JSON errors with an `error` field and an HTTP status.

7. Data Shapes

Function record

```
{
  "id": "billing.py::calculate_discount",
  "file": "billing.py", "name": "calculate_discount", "qualname": "calculate_discount",
  "line_start": 12, "line_end": 18, "is_method": false,
  "signature": "calculate_discount(price: float, discount_percent: float) -> float",
  "docstring": "Return price after applying discount_percent.",
  "source": "def calculate_discount(price, discount_percent):\n    ...",
  "coverage_pct": 0.0, "coverage_label": "untested",
  "last_modified_days": 2, "usage_count": 5,
  "risk_score": 0.84, "risk_label": "High",
  "status": "pending",            // pending | generating | accepted | needs_review | rejected | possible_bug
  "tests": [ /* test records */ ]
}
```

Test record

```
{
  "name": "test_calculate_discount_basic",
  "status": "possible_bug",       // accepted | needs_review | rejected | possible_bug
  "code": "def test_calculate_discount_basic():\n    assert calculate_discount(100, 10) == 90",
  "attempts": 1,
  "reason": "Expected 90, function returned 99",
  "expected": "90", "actual": "99",
  "traceback": "... trimmed to last 15 lines ...",
  "verdict": {"verdict": "likely_bug", "reason": "Function ignores discount_percent."}
}
```

Summary (drives the summary cards and the PDF)

```
{
  "coverage_before": 45.0, "coverage_after": 68.0, "coverage_delta": 23.0,
  "untested_before": 12,   "untested_after": 2,
  "functions_processed": 12,
  "tests_generated": 38, "accepted": 29, "needs_review": 3, "rejected": 6,
  "possible_bugs": 3,
  "newly_covered": ["billing.py::calculate_discount", "cart.py::cart_total"],
  "bugs": [ {"function_id": "...", "test": "...", "expected": "...", "actual": "...", "reason": "..."} ]
}
```

8. Live Progress Feed: Event Protocol

The backend appends every event to `events.jsonl` for the job and streams it over SSE with an incrementing `id`. The browser uses `EventSource`; on reconnect it sends `Last-Event-ID` and the server replays missed events. Event types:

Event	Payload	UI effect
`log`	`{ts, level: info|warn|error|success, message}`	Appends a line to the feed
`stage`	`{stage: parse|coverage|prioritize|generate|validate|report, status: start|done}`	Updates the stepper
`progress`	`{done, total}`	Moves the progress bar
`function_update`	`{function_id, status}`	Updates that function's status chip live
`analysis_complete`	`{summary_preview}`	Shows the Analysis Dashboard
`generation_complete`	`{summary}`	Shows the Summary Report
`error`	`{message, recoverable}`	Shows an error banner

Example feed lines the backend must emit (these exact styles are what users see):

```
[10:42:01] Repository loaded: 14 Python files found
[10:42:03] Parsing code... 38 functions extracted
[10:42:05] Running coverage analysis... current coverage: 45%
[10:42:07] Ranking 12 untested functions by risk
[10:42:09] Generating test for `calculate_discount`...
[10:42:21] Running test for `calculate_discount`... ACCEPTED
[10:42:22] Generating test for `apply_coupon`...
[10:42:35] Running test for `apply_coupon`... POSSIBLE BUG FOUND
[10:42:36] Generating test for `get_user_total`...
```

9. LLM Prompt Templates

Test generation prompt

```
You are an expert Python test engineer. Write pytest unit tests for the function below.

FILE: {file_path}
IMPORT STATEMENT TO USE: {import_stmt}
{class_context}
FUNCTION UNDER TEST:
```python
{function_source}
```
DOCSTRING / INTENT: {docstring_or_"none provided"}
EXAMPLE CALL SITES FROM THE CODEBASE:
{call_sites_or_"none found"}

RULES:
1. Output ONE python code block containing only the test code. No explanation.
2. Use exactly the import statement given. Do not import anything that is not installed.
3. Write 3 to 6 test functions named test_<function>_<scenario>.
4. Cover a normal case, at least one edge case (zero, empty, boundary), and an invalid-input
   case only if the code or docstring shows it raises an error (use pytest.raises).
5. Every test must call the function under test and assert on its real return value
   or on the raised exception.
6. Derive expected values from the docstring/intent and simple hand calculation,
   NOT by guessing what the current code returns.
7. No network, no randomness, no dependence on the current time. Use tmp_path for files.
```

Retry prompt

```
Your previous test code was rejected.
PROBLEM: {specific_error_or_check_failed}
PREVIOUS CODE:
```python
{previous_code}
```
Fix the problem and output ONE corrected python code block. Follow all original rules.
```

Bug verification prompt

```
A generated test failed. Decide whether the FUNCTION has a bug or the TEST is wrong.
FUNCTION:
```python
{function_source}
```
DOCSTRING / INTENT: {docstring}
FAILING TEST:
```python
{test_code}
```
PYTEST OUTPUT (trimmed):
{pytest_output}

Answer with JSON only, no other text:
{"verdict": "likely_bug" | "test_wrong" | "unclear", "reason": "<one sentence>"}
```

10. Frontend Specification

10.1 Screens

Screen	Contents
Home / Input	Upload zip (drag and drop) or GitHub URL field; advanced settings (model dropdown from `/api/models`, max functions, coverage threshold); **Analyze** button; health badge showing Ollama status
Analysis Dashboard	Summary cards (before-state), pipeline stepper, live progress feed, ranked table of candidate functions (file, name, coverage %, last modified, usage count, risk label); row checkboxes; **Generate Tests** and **Generate All** buttons
Generation View	Live progress feed, progress bar, function list with live status chips
Function Detail	Side-by-side code view (feature 9), test list, errors, Copy test button, Regenerate button
Bug Report	Every Possible Bug Found: function, failing test, expected vs actual, reason, link to detail
Summary Report	Summary cards (before and after), per-function table, bug report link, **Download PDF** and **Download accepted tests** buttons

Status chips: green ACCEPTED, orange POSSIBLE BUG FOUND, yellow NEEDS REVIEW, grey REJECTED, blue PENDING or GENERATING (animated). Always pair color with text so it is readable in grayscale.

10.2 The four headline features

Feature A: Side-by-side code view

- Two equal panels. **Left:** the original function with file path and line range in the header. **Right:** the generated test with its status badge in the header.
- Both panels use `react-syntax-highlighter` (Prism, Python), line numbers on, horizontal scroll for long lines, a readable monospace font, and a theme consistent with the app (light and dark).
- If the status is POSSIBLE BUG FOUND, show a highlighted block beneath the panels with: expected value, actual value, the LLM's one-line reason, and the trimmed traceback.
- If a function has several tests, show them as tabs on the right panel. Provide Copy buttons.
- On narrow screens the panels stack vertically.

Feature B: Live progress feed

- A terminal-style panel (dark background, monospace) fed by the SSE `log` events. Each line shows the timestamp, an icon or tag by level, and the message. Color by level (info, warn, error, success).
- Auto-scrolls to the newest line. If the user scrolls up, pause auto-scroll and show a 'Jump to latest' button.
- Shows a slim progress bar (`progress` events) and a stepper (Parse, Coverage, Prioritize, Generate, Validate, Report) driven by `stage` events.
- Reconnects automatically and replays missed events. The feed persists on the page after completion.

Feature C: PDF report export

- A **Download Report** button on the Summary Report screen calls `/api/jobs/{id}/report.pdf`.
- The PDF (reportlab) contains: title, project name, date, model used; summary table (coverage before, after, delta; untested before and after; tests generated, accepted, needs review, rejected; bugs found); the ranked priority list with risk labels; a per-function results table; a Possible Bugs section with function, file and line, test, expected vs actual, and reason; and a limitations note stating that tests are AI-generated and must be reviewed.
- Use only fonts and characters supported by the standard PDF fonts (no emoji) so nothing renders as boxes.

Feature D: Summary cards

- A row of four cards at the top of the Analysis Dashboard and the Summary Report, large numbers with small labels:
- **Coverage**: before to after with the delta, e.g. `45% -> 68% (+23%)`. Before generation it shows the current value only.
- **Untested functions**: e.g. `12 -> 2 remaining`.
- **Tests generated**: total with accepted count, e.g. `38 generated, 29 accepted`.
- **Possible bugs found**: e.g. `3`, in the warning color, clickable to the Bug Report.
- Values come from `GET /api/jobs/{id}/summary` and update live as `function_update` events arrive.

10.3 UX requirements

- Loading, empty and error states for every screen. If Ollama is not running, show a clear message with the command to start it (`ollama serve`) and how to pull the model.
- Disable buttons while a phase is running. Keep the feed visible during runs.
- Accessible: semantic HTML, keyboard focus states, sufficient contrast.

11. Safety and Reliability

- Uploaded repositories and generated tests are untrusted code. Run them only in subprocesses with a timeout and inside the job's workspace, with a minimal environment (do not pass the backend's secrets). State in the README that SmartTest is intended for local use on code the user trusts, and provide an optional Docker setup for stronger isolation.
- Guard against zip-slip, enforce upload size limits and a maximum file count.
- Clean up workspaces older than 24 hours (a simple startup task is enough).
- Handle projects with no tests, no git history, no `requirements.txt`, or failing dependency installs gracefully with warnings in the feed rather than crashes.
- A failure in one function must not stop the whole run: record the error on that function and continue.
- Cap LLM calls with `LLM_TIMEOUT_SEC` (default 120) and handle Ollama being unreachable.

12. Project Structure and Configuration

```
smarttest/
  README.md
  docker-compose.yml            (optional)
  .env.example
  backend/
    app/
      main.py                   FastAPI app, CORS, routers
      config.py                 settings from environment
      jobs.py                   job state, event log, background tasks
      analyzer/                 ast parsing, function extraction, call counting
      coverage_check/           coverage run, JSON parsing, line-to-function mapping
      prioritizer/              git blame recency, usage count, risk score
      generator/                LLMClient, OllamaClient, MockLLMClient, prompts, code extraction
      validator/                static checks, pytest runner, classification, bug verification
      reporter/                 summary builder, PDF (reportlab), tests.zip
    tests/                      unit tests for each module + one end-to-end test
    requirements.txt
  frontend/
    src/ (pages, components, hooks/useEventStream, api client, types)
    package.json, vite.config.ts, tailwind config
  scripts/
    create_sample_repo.py       builds the demo repo with a dated git history at runtime
    sample_repo_template/       committed source files used by that script
  .gitignore                    data/, node_modules/, venv/, .env, __pycache__/, generated sample_repo/
```

Variable	Default	Meaning
`LLM_PROVIDER`	`ollama`	`ollama` or `mock`
`OLLAMA_BASE_URL`	`http://localhost:11434`	Ollama endpoint
`OLLAMA_MODEL`	`qwen2.5-coder:7b`	Model used for generation and verification
`LLM_TIMEOUT_SEC`	120	Per LLM call
`TEST_TIMEOUT_SEC`	30	Per pytest run
`COVERAGE_THRESHOLD`	60	Functions below this percent are candidates
`MAX_FUNCTIONS_DEFAULT`	10	Default number of functions to process
`MAX_UPLOAD_MB`	50	Upload size cap
`WORKSPACE_DIR`	`./data/jobs`	Where job folders live

13. Sample Repository for Demo and Testing

Build a small Python shop project, generated into `sample_repo/` by `scripts/create_sample_repo.py`, which also creates at least 8 git commits with different dates (using `GIT_AUTHOR_DATE` and `GIT_COMMITTER_DATE`) so recency scores differ.

Important for GitHub: the generated `sample_repo/` contains its own `.git` folder, so it must NOT be committed to the SmartTest repository (a nested git repo breaks the parent). Commit only the script and the template files in `scripts/sample_repo_template/`, and add the generated folder to `.gitignore`.

- `billing.py`: `calculate_discount(price, discount_percent)`, `apply_coupon(total, coupon)`, `tax_for(amount, rate)`.
- `cart.py`: a `Cart` class with `add_item`, `remove_item`, `cart_total`.
- `utils.py`: `slugify`, `is_valid_email`, `chunk_list`.
- `tests/` covering roughly 40 percent of the functions (so the baseline is about 40 to 50 percent).
- **Planted bug for the demo:** `apply_coupon` must have a docstring saying the coupon applies when the total is **at least** the minimum amount, but the code uses `>` instead of `>=`. A good generated test for the boundary should fail and be flagged as a Possible Bug Found.
- Include a second, subtle bug (for example `chunk_list` dropping the last partial chunk) if easy.

`MockLLMClient` must return valid, deterministic pytest code for these sample functions, including the boundary test that exposes the planted bug, and a deliberately bad response (for example `assert True`) for one function so the Rejected path is exercised.

14. Build Plan with Acceptance Criteria

Build in this order. Do not start a phase until the previous phase's acceptance criteria pass. Commit after each phase with a clear message, and keep the README updated.

Phase	Deliverable	Acceptance criteria
0 Scaffold	Repo layout, FastAPI app, React app, config, `.env.example`, `/api/health`, mock LLM	Both apps start; health endpoint returns status; frontend shows the Ollama status badge
1 Analyzer	Function extraction and call counting	Sample repo yields the expected functions with correct line ranges, docstrings and signatures; unit tests pass

Phase	Deliverable	Acceptance criteria
2 Coverage	Coverage run and line-to-function mapping	Per-function percentages and total match a manual `coverage report` on the sample repo; works with a repo that has no tests
3 Prioritizer	Git recency, usage counts, risk score and labels	Recently changed, widely used functions rank above old, unused ones on the sample repo; works without git history
4 Generator	LLMClient (Ollama + mock), prompts, extraction, retries	Given a function, returns valid test code; retry path works when the mock returns invalid code
5 Validator	Static checks, pytest runner, classification, bug verification	Sample repo produces at least one each of Accepted, Rejected and Possible Bug Found (the planted bug); flaky and timeout cases become Needs Review
6 Orchestration	Jobs, background tasks, event log, SSE, all REST endpoints	Full two-phase run works through curl; reconnecting with `Last-Event-ID` replays events
7 Frontend	All screens plus features A to D	Complete run in the browser on the sample repo; feed streams live; side-by-side view highlights syntax; cards update
8 Reports	PDF, tests.zip, summary endpoint	PDF opens and contains every section listed in feature C; zip contains only accepted tests
9 Polish	End-to-end test, README, error states, optional Dockerfile	One command each to start backend and frontend; README explains install, Ollama setup, demo steps; end-to-end test passes with the mock LLM
10 Delivery	Everything pushed to the GitHub repository (see below)	All code is visible on the `main` branch of the repository, a fresh clone installs and runs by following the README

Delivery to GitHub

- Target repository: **https://github.com/Baji-Shaida/Smart-Test**. After the project is complete, push all work to this repository's `main` branch.
- Commit after every phase during the build and push regularly, so progress is visible and nothing is lost. If the repository already contains files (for example a default README), merge with them and do not overwrite or delete anything without need.
- Add a `.gitignore` before the first commit: `data/`, `node_modules/`, `venv/`, `.venv/`, `.env`, `__pycache__/`, `*.pyc`, build output, and the generated `sample_repo/`. Commit `.env.example` only; never commit secrets, uploaded projects, job workspaces or model files.
- The repository root must contain the README, `backend/`, `frontend/`, `scripts/` and this specification (as `docs/SmartTest_Technical_Specification.pdf`).
- Finish by verifying a fresh clone works: clone, follow the README, run the demo on the sample repo. Report the final commit link.

Optional stretch goals (only after phase 9)

- Mutation testing badge: inject small mutations (swap `+` and `-`, change `>` to `>=`, flip a condition) into the target function and re-run each accepted test; label tests Strong, Moderate or Weak by how many mutants they catch.
- Coverage heatmap of the file tree, confidence score per test, history page of past runs, one-click copy of a GitHub PR comment.

15. Known Limitations and Definition of Done

Limitations (also shown in the PDF report)

- The LLM infers intent from code and docstrings; it cannot know what the developer meant. Poorly named or undocumented functions yield weaker tests.
- Possible Bug Found means 'worth a human look', not proof of a bug.
- Static analysis and subprocess runs do not reproduce every runtime condition (network, concurrency, external services).
- Functions needing complex fixtures or external services may be Rejected or Needs Review.

Definition of done

- All code is pushed to https://github.com/Baji-Shaida/Smart-Test (`main` branch) with a clear commit history.
- The sample repo demo runs end to end in the browser on a laptop with Ollama and a 7B model, showing the live feed, summary cards, side-by-side view with a flagged bug, and a downloadable PDF.
- All backend unit tests and the end-to-end test pass; frontend builds without errors.
- README documents setup, configuration, the demo script, limitations, and the safety note.
- No paid service or API key is required anywhere.